Fine-tuning is often short and changes the model to a smaller extent than pre-training. Fine-tuning for longer results in forgetting the pretrained information and harms the performance (Gekhman et al., 2024). Since fine-tuning loss is often smooth and has a bounded curvature, we solve the above problem by generating a synthetic data that matches the gradient of real data at the pretrained parameters. We prove that training on such a subset converges to a close neighborhood of the solution found by training on real data.

Challenges of Readable Text Generation. Solving Problem 3 is very challenging, as the set of feasible solutions is sparse, the space is discrete, and LLMs are non-linear and high-dimensional. Specifically, the constraint set is formed by the Cartesian product of many discrete sets, each restricting a word to belong to the vocabulary. Among sequences that satisfy this condition, only those that are readable—measured by a low perplexity value—are valid. Thus, solving Problem 3 is NP-hard as it requires going through all the possible sequences of words in the vocabulary and finding readable sequences that best match the real data gradient. The number of such sequences is exponential in the size of the vocabulary. This makes it computationally infeasible to find the optimum solution. Finally, calculating the similarities in the gradient space of LLMs with billions of parameters is computationally very expensive.

4.2. Alternating Between Text and Embedding Spaces

To solve the above discretely constrained non-convex optimization problem, we first transfer it to the continuous embedding space, where one can optimize the embeddings of synthetic data to match the target gradient, under the constraint that the optimized embeddings belong to the set of all token (words, subwords, or characters) embeddings in the vocabulary. If such embeddings can be found, they can be directly mapped to a sequence of words in the vocabulary.

Formally, let $\boldsymbol{x} \in \mathbb{R}^{n\times d}$ be the embedding matrix of a synthetic sample s with n tokens, where row $x_j \in \mathbb{R}^d$ is the j^{th} token embedding. By stacking the embedding matrices of all synthetic samples in $\mathcal{D}_{\text{syn}}$, we obtain an embedding tensor $\boldsymbol{X} \in \mathbb{R}^{|\mathcal{D}_{\text{syn}}|\times n\times d}$. With an abuse of notation, we denote $\ell(\boldsymbol{x},\boldsymbol{\theta}) = \ell(s,\boldsymbol{\theta})$ and $\ell(\boldsymbol{X},\boldsymbol{\theta}) = \frac{1}{|\mathcal{D}_{\text{syn}}|}\sum_{i=1}^{|\mathcal{D}_{\text{syn}}|}\ell(\boldsymbol{x}^i,\boldsymbol{\theta})$. We rewrite Problem 3 as:

$$\underset{\substack{\mathcal{D}_{\text{syn}},|\mathcal{D}_{\text{syn}}|\le r,\\ x_j\in\mathcal{E},\text{ppl}(\boldsymbol{x})\le\epsilon\\ \forall\boldsymbol{x}\in\mathcal{D}_{\text{syn}}}}{\arg\min} f(\boldsymbol{X}) \;\text{ s.t. }\; f(\boldsymbol{X}) = D(\nabla_{\boldsymbol{\theta}}\ell(\boldsymbol{X},\boldsymbol{\theta}),\nabla_{\boldsymbol{\theta}}\ell(\mathcal{D}_{\text{real}},\boldsymbol{\theta})), \tag{4}$$

where $\mathcal{E} = \{e_1, e_2, \ldots, e_{|V|}\}$ denote the vocabulary embedding, i.e. the set of all token embeddings in the vocabulary V of model $\boldsymbol{\theta}$ where $e_i \in \mathbb{R}^d$ and d is the embedding dimension.

To solve the above constrained optimization problem we apply the Alternating Direction Method of Multipliers (ADMM) (Glowinski & Marroco, 1975; Gabay & Mercier, 1976). By forming the augmented Lagrangian function, ADMM decomposes the original problem into subproblems that can be solved separately and iteratively.

While ADMM was originally introduced for convex optimization under linear constraints, more recently it has been successfully applied to solving mixed integer non-linear programs (Leng et al., 2018; Lin et al., 2019), with convergence guarantees (Huang et al., 2021).

Constrained Gradient Matching in the Embedding Space.

To apply ADMM to our discretely constrained non-convex problem 4, we convert it to a non-convex optimization with convex linear constraints. To do so, we introduce an auxiliary variable $\boldsymbol{Z}$ and rewrite our objective with an extra equality constraint so that the embeddings are constrained to be from the vocabulary, but not subject to that restriction:

$$\min_{\boldsymbol{X}} f(\boldsymbol{X}) + \mathcal{I}_{\mathcal{E}}(\boldsymbol{Z}), \quad s.t. \quad \boldsymbol{X} = \boldsymbol{Z}. \tag{5}$$

The indicator function $\mathcal{I}_{\mathcal{E}}(\boldsymbol{Z})$ is defined as $\mathcal{I}_{\mathcal{E}}(\boldsymbol{Z}) = 0$ if $z_j \in \mathcal{E}\ \forall j$ (i.e., if the embedding of each synthetic example can be mapped to a sequence of words in the vocabulary), and $\mathcal{I}_{\mathcal{E}}(\boldsymbol{Z}) = +\infty$ otherwise. The augmented Lagrange of Eq. 5 for parameter $\rho > 0$, can be formulated as:

$$\mathcal{L}_{\text{aug}}(\boldsymbol{X},\boldsymbol{Z},\boldsymbol{\Lambda}) = f(\boldsymbol{X}) + \mathcal{I}_{\mathcal{E}}(\boldsymbol{Z}) + \langle\boldsymbol{\Lambda},\boldsymbol{X}-\boldsymbol{Z}\rangle + \frac{\rho}{2}\|\boldsymbol{X}-\boldsymbol{Z}\|^2, \tag{6}$$

where $\boldsymbol{\Lambda} \in \mathbb{R}^{|\mathcal{D}_{\text{syn}}|\times n\times d}$ denotes the Lagrangian multipliers. With simple algebraic manipulations, Eq.6 can be written as:

$$\mathcal{L}_{\text{aug}}(\boldsymbol{X},\boldsymbol{Z},\boldsymbol{\Lambda}) = f(\boldsymbol{X}) + \mathcal{I}_{\mathcal{E}}(\boldsymbol{Z}) + \frac{\rho}{2}\|\boldsymbol{X}-\boldsymbol{Z}-\rho^{-1}\boldsymbol{\Lambda}\|^2. \tag{7}$$

ADMM solves the above problem by minimizing primal variables $\boldsymbol{X},\boldsymbol{Z}$ and maximizing dual variable $\boldsymbol{\Lambda}$ at each iteration t, using the following update rules:

$$\text{Primal update:}\quad \boldsymbol{X}^{t+1} = \arg\min_{\boldsymbol{X}} \mathcal{L}_{\text{aug}}(\boldsymbol{X},\boldsymbol{Z}^t,\boldsymbol{\Lambda}^t), \tag{8}$$

$$\boldsymbol{Z}^{t+1} = \arg\min_{\boldsymbol{Z}} \mathcal{L}_{\text{aug}}(\boldsymbol{X}^{t+1},\boldsymbol{Z},\boldsymbol{\Lambda}^t), \tag{9}$$

$$\text{Dual update:}\quad \boldsymbol{\Lambda}^{t+1} = \boldsymbol{\Lambda}^t + \rho(\boldsymbol{X}^{t+1}-\boldsymbol{Z}^{t+1}), \tag{10}$$

which are respectively the proximal step, projection step, and dual update. The proximal step optimizes the embeddings to match the target gradient, and the projection step maps the embeddings to words in the vocabulary. Eq. 8 requires solving an unconstrained optimization problem. When ρ is large, the function is strongly convex in $\boldsymbol{X}$. In practice, stochastic gradient descent algorithms such as Adam (Kingma, 2014) can obtain an approximate solution, which is sufficient for the convergence of ADMM (Huang et al., 2021). Next, we discuss the projection step.

Algorithm 1 GRADient matching w. ADMM (GRADMM)

Input: Constant $\rho > 0$, ADMM steps T, proj param k, DP param ε, δ
Step 1: Initialization
Random sample $\boldsymbol{X} \in \Gamma$
Initialize $\boldsymbol{X}^0 = \arg\min_{\boldsymbol{X}} f(\boldsymbol{X}, \varepsilon, \delta)$
Initialize $\boldsymbol{Z}^0 = \boldsymbol{X}^0$ and $\boldsymbol{\Lambda}^0 \in \mathbb{R}^{|\mathcal{D}_{\text{syn}}| \times n \times d}$.
Step 2: ADMM
for $t = 0, 1, \ldots, T-1$ **do**
  Update $\boldsymbol{X}$: $\boldsymbol{X}^{t+1} = \arg\min_{\boldsymbol{X}} \mathcal{L}(\boldsymbol{X}, \boldsymbol{Z}^t, \boldsymbol{\Lambda}^t, \varepsilon, \delta)$
  Update $\boldsymbol{Z}$: $\boldsymbol{Z}^{t+1} = \mathcal{P}_{\mathcal{E}_{\text{top-k}}}(\boldsymbol{X}^{t+1} + \rho^{-1}\boldsymbol{\Lambda}^t)$
  Update $\boldsymbol{\Lambda}$: $\boldsymbol{\Lambda}^{t+1} = \boldsymbol{\Lambda}^t + \rho(\boldsymbol{X}^{t+1} - \boldsymbol{Z}^{t+1})$
end for
$\mathcal{S} = \mathcal{P}_{\mathcal{E}_{\text{top-k}}}(\boldsymbol{X}^T)$
Step 3: Filtering
Drop samples in $\mathcal{S}$ that do not belong to their category
Select r samples in $\mathcal{S}$ with lowest gradient matching loss
Drop examples with highest loss from categories that have a higher average gradient matching loss
Output: Remaining synthetic texts in $\mathcal{S}$.

Projecting the Embeddings into the Vocabulary Space. Equation (9) can be written as (Huang et al., 2021):

$$\boldsymbol{Z}^{t+1} = \arg\min_{\boldsymbol{Z}} \mathcal{L}_{\text{aug}}(\boldsymbol{Z}^{t+1}, \boldsymbol{Z}, \boldsymbol{\Lambda}^t) \tag{11}$$

$$= \arg\min_{\boldsymbol{Z}} \mathcal{I}_{\mathcal{E}}(\boldsymbol{Z}) + \|\boldsymbol{Z} - \boldsymbol{X}^t - \rho^{-1}\boldsymbol{\Lambda}^t\|^2 \tag{12}$$

$$= \mathcal{P}_{\mathcal{E}}(\boldsymbol{X}^t + \rho^{-1}\boldsymbol{\Lambda}^t). \tag{13}$$

For the vocabulary embeddings $\mathcal{E}$, the projection $\mathcal{P}_{\mathcal{E}}(x_i)$ of an embedding vector $x_i \in \mathbb{R}^d$ into the vocabulary space is the embedding vector $z_i := \arg\min_{e \in \mathcal{E}} \|x_i - e\|^2$ corresponding to the token in the vocabulary that is closest to x_i in Euclidean space. In practice, z_i can be found by looping over the vocabulary and finding the closest token. This operation can be vectorized efficiently. For an embedding matrix $\boldsymbol{x} \in \mathbb{R}^{n \times d}$ consisting of n embedding vectors, we project each embedding vector x_i independently to get the matrix embedding $\mathcal{P}_{\mathcal{E}}(\boldsymbol{x}) = \left[\mathcal{P}_{\mathcal{E}}(x_1) \;\; \mathcal{P}_{\mathcal{E}}(x_2) \;\; \cdots \;\; \mathcal{P}_{\mathcal{E}}(x_n)\right]^\top$. Similarly, for the embedding tensor $\boldsymbol{X}$, the projection operation can be vectorized efficiently to find $\boldsymbol{Z}$.

Ensuring Readability of the Projected Text. Projecting embeddings to tokens in vocabulary independently does not yield meaningful text. To address this, we leverage the idea of top-k decoding to enforce the readability of generations (Fan et al., 2018). Consider an embedding matrix $\boldsymbol{x} \in \mathbb{R}^{n \times d}$ consisting of n embedding vectors. For every embedding x_i, we find the top k most probable tokens from the vocabulary condition on the previously projected tokens. Formally, we find the top k tokens that minimize $\sum_{e \in \mathcal{E}} P(x | x_{i=1:i-1})$, and denote them by $\mathcal{E}_{\text{top-k}}$. Then, we project x_i into the space of the top-k selected tokens by solving $z_i := \mathcal{P}_{\mathcal{E}_{\text{top-k}}}(x_i) = \arg\min_{e \in \mathcal{E}_{\text{top-k}}} \|x_i - e\|^2$.

4.3. Dealing with High-dimension Gradients

Calculating similarities in the very high-dimensional gradient space of LLMs with billions of parameters is computationally very expensive. Besides, such gradients contain many small and noisy dimensions which makes calculating gradient similarities inaccurate. An effective way to tackle this issue is to leverage lower-dimensional gradient estimates (Mirzasoleiman et al., 2020). Various weight initialization (Glorot & Bengio, 2010) and activation normalization methods (Ioffe, 2015) uniformize the activations across samples. Thus, the variation of the gradient norm is mostly captured by the gradient of the loss with respect to the model's last layer (Katharopoulos & Fleuret, 2018).

Based on the observation, we generate synthetic data by only matching the last-layer gradient of the model. Let $\boldsymbol{\theta}_L$ denote the last layer of model $\boldsymbol{\theta}$ with L layers, the last-layer gradient distance between synthetic and real data in Eq 4 is:

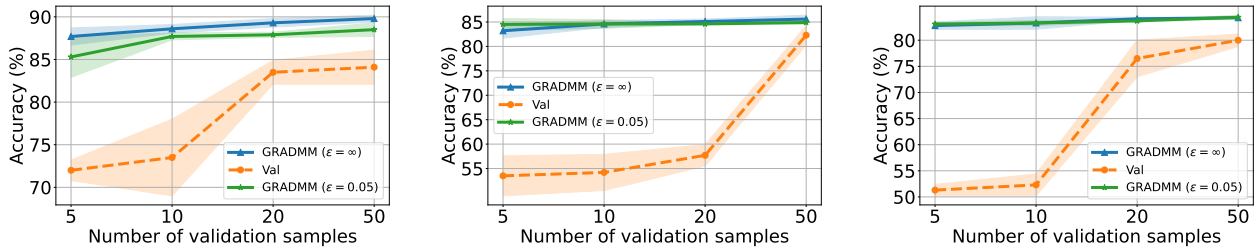
$$\arg\min_{\substack{\mathcal{D}_{\text{syn}}, |\mathcal{D}_{\text{syn}}| \le r, \\ x_j \in \mathcal{E}, \text{ppl}(\boldsymbol{x}) \le \epsilon \\ \forall \boldsymbol{x} \in \mathcal{D}_{\text{syn}}}} D(\nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{X}, \boldsymbol{\theta}_L), \nabla_{\boldsymbol{\theta}} \ell(\mathcal{D}_{\text{real}}, \boldsymbol{\theta}_L)) \tag{14}$$

Matching the last layer gradient is much cheaper than the full gradient and allows generating synthetic data with superior performance, as we will confirm in our experiments.

4.4. Filtering the Generated Examples

While the top-k projection enables generating human-readable text, it can negatively affect the performance due to the following reasons: (i) It may change the category of the synthetic example by including words that are most relevant to other categories. (ii) It may significantly increases the gradient matching loss of some synthetic examples. (iii) It may result in a much higher gradient matching loss for some categories compared to the rest. To address the above issues, we filter the low-quality synthetic examples as follows. First, we drop examples that do not belong to the correct category by running a simple few-shot evaluation (Li et al., 2023b), as detailed in Appendix B. Next, for every category we select r synthetic examples with the lowest gradient matching loss. Finally, we ensure similar gradient matching loss for all categories by dropping examples with highest loss in categories with a higher average loss compared to the rest. The above steps significantly boost the performance of the synthetic data, as we will confirm in our experiments.

Remark. Due to top-k decoding, the filtered synthetic examples do not match the target gradient very accurately. Thus, we generate each synthetic example to match the target gradient independently, not conditioned on each other. We will confirm in our experiments in Sec. 5.3 that the synthetic data generated independently by GRADMM matches the real data gradient closely during fine-tuning.



(a) SST-2 (b) Tweet emotions (c) Rotten tomatoes

Figure 1. Data-scarce regime. Generating 100 synthetic samples with GRADMM, based on 5, 10, 20, 50 examples from a target task. Synthetic data generated based on only 5 real examples outperforms the real data by 15.7%, 29.7%, and 31.5% on the three datasets.

4.5. Making GRADMM Differentially Private

Differential privacy (DP) (Dwork et al., 2006) is a rigorous mathematical framework that ensures no single data point can be identified or inferred from the output of a statistical or machine learning model. To make GRADMM differentially private, we inject controlled noise α into the clipped gradient of the real data in Equation (4). Specifically, GRADMM first computes per-sample gradients of the real data and clips their ℓ_2-norm to a threshold of C. These clipped gradients are then averaged, and Gaussian noise, drawn from $\mathcal{N}(0, \sigma^2)$, is added to this average. The added noise scale σ is defined as follows:

$$\sigma = \begin{cases} \frac{C\sqrt{2\log\frac{1.25}{\delta}}}{\varepsilon|\mathcal{D}_{\text{real}}|}, & \text{if } 0 < \varepsilon \leq 1 \text{ (Dwork et al., 2014)} \\ \frac{C(c+\sqrt{c^2+\varepsilon})}{\sqrt{2}\varepsilon|\mathcal{D}_{\text{real}}|}, & \text{if } \varepsilon > 1 \text{ (Lowy \& Razaviyayn, 2021)} \end{cases}$$

where $c = \sqrt{\log\left(\frac{2}{\sqrt{16\delta+1}-1}\right)}$ and the clipping threshold $C = 1$ for all experiments. Based on the composition theorem (Dwork et al., 2010), GRADMM achieves (ε, δ)-DP.

We denote the new optimization problem and its augmented Lagrangian objective as $f(X, \varepsilon, \delta)$ and $\mathcal{L}(\boldsymbol{X}, \boldsymbol{Z}, \boldsymbol{\Lambda}, \varepsilon, \delta)$, respectively. As $\varepsilon \to \infty$, the privacy constraint is relaxed and $f(X, \varepsilon, \delta) \to f(X)$, yielding the original optimization problem. The new problem retains the same structure and can be solved using the ADMM procedure described before.

Pseudocode of our method,GRADMM, is illustrated in Alg1.

4.6. Convergence Analysis

Next, we theoretically analyze the convergence of fine-tuning on the synthetic examples generated by GRADMM. As discussed in Sec. 3, fine-tuning is short and changes the model to a small extent compared to pretraining. Effectively, the fine-tuning loss is relatively flat and can be modeled by a β-smooth (i.e., with a bounded Hessian $\boldsymbol{H}$) and μ-PL* $\big($i.e., $\|\nabla\mathcal{L}(\boldsymbol{\theta})\|^2 \geq 2\mu\mathcal{L}(\boldsymbol{\theta})\big)$ function.

For a synthetic subset generated by GRADMM via matching the gradient of real data at the pretrained model parameters, the following lemma bounds the error between the gradient of synthetic and real data during the fine-tuning.

Lemma 4.1. *Assume that the fine-tuning losses of the real $\mathcal{L}$ and synthetic data $\mathcal{L}^s$ are β-smooth. The synthetic data generated by* GRADMM *that captures the gradient of real data by an error of $\|\nabla\mathcal{L}(\boldsymbol{\theta}) - \nabla\mathcal{L}^s(\boldsymbol{\theta})\| \leq \epsilon$ at the pretrained parameters $\boldsymbol{\theta}$, has a bounded gradient error at any point t during fine-tuning:*

$$\|\nabla\mathcal{L}(\boldsymbol{\theta}_t) - \nabla\mathcal{L}^s(\boldsymbol{\theta}_t)\| \leq 2\beta\delta + \epsilon, \tag{15}$$

where $\delta \geq \|\boldsymbol{\theta} - \boldsymbol{\theta}_t\|$ upper-bounds the norm of change to the parameteres during fine-tuning.

Next, we analyze the convergence of fine-tuning with gradient descent on the synthetic subset generated by GRADMM.

Theorem 4.2. *For a μ-PL* loss function $\mathcal{L}$, under the assumptions of Lemma 4.1, gradient descent on the synthetic data converges with the same rate as that of real data. Moreover, at every step t, the difference between the fine-tuning loss on synthetic and real data is upper bounded by:*

$$|\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}^s(\boldsymbol{\theta}_t)| \leq \xi(2\nabla - \xi)/2\mu. \tag{16}$$

where $\xi = 2\beta\delta + \epsilon$ and ∇ is an upper bound on the gradient norm during fine-tuning.

The next corollary shows that fine-tuning on real data and synthetic data found by GRADMM yields similar models.

Corollary 4.3. *Consider a strongly convex loss (i.e., $\|\boldsymbol{H}\| \geq \alpha > 0$) with unique minimizer $\boldsymbol{\theta}_*$ and let $\mathcal{L}(\boldsymbol{\theta}_*) = 0$. Then fine-tuning with any optimizer on real and synthetic data generated by* GRADMM *yield similar models:*

$$\|\boldsymbol{\theta}_* - \boldsymbol{\theta}^s_*\| \leq \sqrt{\xi(2\nabla - \xi)/\alpha\mu}. \tag{17}$$

Thus, the fine-tuned models will have a similar performance.

5. Experiments

5.1. Experimental settings

Datasets. We apply GRADMM to different text classification datasets including SST-2 movie reviews (Socher et al.,